

NexusAI: LLM Integration & Dynamic Query Defense Brief

2026-09-17 · prepared by @Someone

Executive summary

The last two weeks (2026-09-03 → 2026-09-17) were spent on two hard problems: getting a local LLM to reliably help answer investigator questions, and making the system answer questions that aren't covered by any predefined template or operation. Neither is a "quick feature" — both required building real infrastructure and repeatedly proving, live, that each attempt either worked or didn't.

Headline result: the dynamic runtime query engine — the ability to answer a question like *"how many CDR records do we have"* or *"which department has the highest average invoice total"* with no predefined template — now works, is fast (under 300ms), and is backed by real parameterized SQL over the full dataset, not a sample. This is demo-ready today.

Headline honesty: the LLM narration role (turning a fact packet into natural prose) is not yet fast enough to demo confidently for open-ended "explain this" questions — it can take 100+ seconds and sometimes still falls back to a deterministic answer. That fallback is working exactly as designed (it never serves an ungrounded answer), but the latency itself is unresolved. This brief says so plainly rather than papering over it.

What happened in today's session specifically: while preparing this brief, three more things were found and fixed live — a critical authentication-bypass bug that had been silently active on every rebuild this session, a Dockerfile that had been silently running the container without several required packages for 45 hours, and a same-family model quantization swap that measurably fixes the worst of the narration latency (see below). All three are live-verified, not just claimed.

Critical fixes shipped today

Issue	Severity	Root cause	Fix	Live verification
Authentication bypass	Critical	<code>docker-compose.forensic-records.yaml</code> defaulted <code>FORENSIC_API_AUTH_REQUIRED</code> to <code>false</code> and the API key to empty whenever the real values weren't exported before <code>docker compose up</code> — which had been silently true for every rebuild-and-verify cycle this session	Flipped the compose default to <code>fail-closed (true)</code> ; the Go-level guard (<code>validateForensicAuthConfig</code>) that enforces this was already written and tested, only the compose default was wrong	Confirmed twice live: key unset → container refuses to start (crash-loop, loud error) instead of serving unauthenticated traffic; key set → unauthenticated request returns 401, authenticated request returns 200
Dockerfile package drift	High	<code>api/forensic_records/Dockerfile</code> only copied <code>pkg/httpclient</code> , silently omitting <code>pkg/forensicpolicy</code> , <code>pkg/forensicrequest</code> , <code>pkg/forensictext</code> — the running container had been missing packages current source needed for roughly 45 hours	Changed <code>COPY pkg/httpclient ./pkg/httpclient</code> to <code>COPY pkg ./pkg</code>	Rebuilt image, full suite still 711/711, container starts clean
Narration model too slow	High	Default synthesis model was the Q8_0 build of Qwen3-4B (4.28GB) — on this CPU-only host, memory-bandwidth-bound inference made it too slow for interactive use	Switched the default to <code>qwen3-4b-instruct-2507-q4km-nxb21d-dev</code> , the Q4_K_M quantization of the same model (2.49GB, same tokenizer/template/family already validated across the narration test suite)	See benchmark below — measured live against the real narration JSON-schema payload, not simulated

The model benchmark, run head-to-head today against the exact JSON-schema narration payload the production narration path sends (facts + citations, `max_tokens: 250`, `temperature: 0`):

Model	Size	Result
<code>qwen_qwen3-4b-instruct-2507</code> (Q8_0) — the model live in production all session	4.28 GB	Did not finish generating a 250-token response within 240 seconds
<code>qwen3-4b-instruct-2507-q4km-nxb21d-dev</code> (Q4_K_M, same family)	2.49 GB	Finished in 72 seconds
<code>phi4-mini-instruct-nxb21d-semantic</code>	2.49 GB	Finished in 64 seconds (complete answer, no truncation)

This is the single most concrete piece of evidence for "why narration has been stuck for two weeks": the model that was actually deployed in production this entire time could not complete a realistic request inside 4 minutes. Switching to the Q4_K_M build of the same Qwen3-4B model — not a new model family — is the safe, same-day fix: identical

tokenizer, prompt template, and grounding behavior already proven across 711 passing tests, versus Phi-4-mini which is a different model family that would need its own accuracy/grounding validation pass before being trusted the night before a demo. Full rationale for not switching to Phi-4-mini is in the LLM section below.

Both `.env.forensic-runtime.local` and `docker-compose.forensic-records.yaml` were updated and the container was redeployed and re-verified live (auth still fail-closed, correct model baked into the running container's env).

Two-week timeline (2026-09-03 → 2026-09-17)

Reconstructed from the running engineering log (`NEXUSAI_CONTINUATION.md`). Two work streams ran in parallel: local-LLM model qualification (which model, if any, is good enough for which role) and dynamic-query engineering (answering questions no template covers). A third, shorter stream simplified the Investigation Workspace UI (Sep 3–8).

Date	Work item	Verified outcome
Sep 3	NX-B2.1 reconciliation + source closures; Qwen 4B pre-model RAM gate check	Reconciliation matched live state; RAM gate correctly blocked loading a second model (3.37GB free) — nothing shipped yet
Sep 3	Video/Activity live replay found 2 real defects, then fixed	Live-verified PASS: 4 fresh queries all returned exact correct rows
Sep 4	Offline evaluation harness hardened through 5 infra bugs (RAM-cache pressure, wrong ingest enum, over-strict dirty-page check)	Final dry run passed all gates without touching the model
Sep 4	Qwen 4B one-shot 168-case holdout run	Invalidated: 80 malformed + 1 timeout, 0 passes — root cause was a config mismatch (context size, temperature), not model quality. Holdout permanently retired regardless
Sep 4	Found and fixed real Windows/PowerShell transport bugs (BOM handling, multilingual text corruption, serialization depth)	31/32 on a corrected 32-case corpus; 1 failure was a genuine model defect (split an identifier <code>ISB-5907</code> into two tokens)

Date	Work item	Verified outcome
Sep 5	Fixed the identifier-splitting defect server-side; found 3 new failures in an unrelated path (homoglyph substitution, false clarifications)	13/16 — candidate marked <code>DEVELOPMENT_FAIL_DO_NOT_ADVANCE</code>
Sep 5	Fixed phrase-literal parsing (quoted spans, Unicode)	19/20 on a fresh corpus
Sep 5	Q4 168-case final qualification attempt	Invalidated after 2/168 — both truncated at 192 tokens; production's schema was larger than the dev schema that had passed. Budget raised 192→512. Holdout permanently retired
Sep 6	First deterministic-first/LLM-residual hybrid attempt	Incomplete failure on the very first LLM case (schema bug); fixed
Sep 7	Decision8 adjudication (8-case gate) run live	5/8 pass, 3/8 fail on clarification-weakness and multilingual-intent-weakness → formal decision: current 4B model RETIRED for full-plan use, insufficient for residual role too. Selected Qwen3-8B as next candidate
Sep 7–8	Investigation Workspace UI redesigned three times (unified → simplified → premium drawer), each live-verified	45–56 unit tests, 27–29 Playwright scenarios per pass; each pass caught and fixed one real live-review bug before shipping
Sep 8	Qwen3-8B selection gate dispatched twice	Both resource-blocked (RAM floor), not quality failures — explicitly logged as <code>INSUFFICIENT_EVIDENCE</code> , not a verdict
Sep 9	Full-registry semantic planner foundation built (server stays authoritative on scope/SQL/facts, model only ranks)	527 specs passing; source-validated, not yet live-tested
Sep 9	Qwen3-8B resource-proof run reached the first real request before RAM dropped	Consumed with zero completed cases — <code>INSUFFICIENT_EVIDENCE</code> again

Date	Work item	Verified outcome
Sep 10	45-case English product-proof ran to completion	Genuine functional failure, not resource: planner 0/16 correct operation selections; narration 0/13 (100% fallback) — root cause found: an internal 8-second cap had never actually been raised to the configured 120s, contradicting an earlier claim that it already was
Sep 10	Fixed the 8-second cap; fixed 2 real hallucinations in general-assistant answers (invented OCR confidence thresholds, invented licensing claims)	New cancellation/timeout tests added
Sep 11	48-case V2 product-proof, 40/48 completed before a wall-time guard	Operation selection 0/14 again; narration 1/14 validated + 13 safe grounding-fallbacks (explicitly logged as fallbacks, not hallucinations) — corpus consumed, do not rerun
Sep 11	Hierarchical family→operation selection + conservative grounded paraphrase built	Source-tested; live SQL execution explicitly left open for later
Sep 14	Source-native typed SQL algebra shipped — the foundation of today's dynamic-query fix	Live example: "which department has the highest average invoice total" → correct answer (<code>Sales / 400</code>) against an independent oracle
Sep 14	Retrieval-first semantic resolver (BM25 over 66 operations, model picks from top-5)	137-query ledger: top-1 85.4%, top-3 92.0%, top-5 96.35%, stable across 10 runs
Sep 15	Conversational request-class contract shipped	Full suites passed
Sep 15	Fair current-4B adjudication: 5 completed cases, model chose "dynamic plan" instead of the correct registered operation in all 5	0/5 → triggered formal decision <code>CURRENT_4B_SEMANTIC_ROLE=REPLACE_REQUIRED</code>

Date	Work item	Verified outcome
Sep 15	Phi-4-mini acquired and qualified live end-to-end for the semantic-selection role	Failed: 0/5 registered, 0/2 dynamic; p95/max latency 180,019ms — 6x over the 30s limit. Decision: stop, do not tune, do not acquire a third candidate without approval
Sep 16	Embedding-assisted ranking shipped (qwen3-embedding-0.6b used only for ranking signal, never authorization)	137-query ledger top-1 improved 130→131/137, zero regressions, 122/122 correct where confident
Sep 16	NX-B2.1D baseline closure	RAM-gated model adjudication explicitly left <code>PREPARED_NOT_EXECUTED</code> — no model verdict claimed without evidence
Sep 17	Query-understanding investigation found the model-based operation-chooser had zero production callers (deliberately retired); re-enabled as a bounded fallback, reproduced a live wrong answer, root-caused it as a pre-existing deterministic bug , fixed the underlying vocabulary gap	Confirmed live, twice, with the LLM fully disabled both times to isolate the real cause
Sep 17	Dynamic SQL executor built — see next section	"How many CDR records" now returns the exact DB count in under 300ms, verified against <code>SELECT count(*)</code>
Sep 17	Extended the SQL fix to IPDR, fixed a real HTTP 500, fixed a systemic keyword-routing ordering bug behind 3 separate collision bugs, added an intent-compatibility gate, fixed a document/OCR/transcript relevance-filtering gap	Full suite 711/711 re-confirmed after each fix

Date	Work item	Verified outcome
Sep 17	LLM fallback re-attempted a second time, reproduced a wrong answer again, root-caused as the <i>same</i> pre-existing deterministic gap (not the LLM)	Fixed the vocabulary gap; explicit recommendation logged to stop re-attempting the LLM fallback and build a general embedding-grounding signal instead of patching keywords one at a time
Sep 17 (today)	Auth bypass, Dockerfile drift, and the Q4_K_M model swap — see previous section	All three live-verified

The pattern across two weeks is consistent: every model-qualification attempt that looked like a pass was checked for whether it was actually resource-starved rather than quality-tested, and every "fixed" dynamic-query claim was re-verified against the real database before being written down as done.

LLM model integration: every model, every role, every result

The system has (at least) two distinct places an LLM could be used, and they behave very differently:

1. **Query understanding / operation selection** ("which of the 66 registered operations answers this question") — every model tried here has failed or been rolled back.
2. **Narration** ("turn a computed fact packet into readable prose with citations") — this one works correctly when it isn't timing out, and falls back safely when it can't finish in time.

Models evaluated, by role

Model	Role tried	Verdict
qwen_qwen3-4b-instruct-2507 (Q8_0)	Full planning → residual/hybrid fallback → narration	Full-plan: RETIRED (Decision8, 5/8, failed on clarification and multilingual weakness). Registered-operation selection: REPLACE_REQUIRED (0/5, always chose "dynamic plan" over the correct registered op). Narration: works, but too slow in the Q8_0 build — fixed today by quantization swap

Model	Role tried	Verdict
qwen3-4b-instruct-2507-q4km (dev variants)	Development candidate for the roles above	Extensively iterated; fixed a real identifier-splitting bug, a completion-truncation bug, a homoglyph-substitution bug. Best results 19/20 and 16/16 on narrow corpora, but the 168-case independent qualification holdout was permanently invalidated by the schema/budget bug before a full verdict could be reached
qwen3-8b-q4km (two selection-dev variants)	Candidate replacement for the retired 4B role	Never got a quality verdict at all — every dispatch was consumed by RAM/resource failure before enough cases completed. Final state: <code>INSUFFICIENT_EVIDENCE</code> , not a rejection on merit
phi4-mini-instruct-nxb21d-semantic	Registered-operation selection only	Qualified live end-to-end: failed — 0/5 registered, 0/2 dynamic; p95/max latency 180,019ms, 6x over the 30s limit. Decision: stop, do not tune, do not acquire a third candidate without explicit approval
qwen3-embedding-0.6b	Ranking signal only (never authorization/execution) inside the deterministic compiler	Worked: measurable, small, positive improvement (130→131/137 top-1) with zero regressions

Why query-understanding/planning attempts kept failing

Three independent models, three independent failures, and the *shape* of the failure is consistent: given a question that has a correct, registered, exact-match operation, the model chooses a generic "dynamic plan" or the wrong operation instead of the specific one that exists. This isn't a one-off — it happened with Qwen 4B (0/5), and Phi-4-mini (0/5 registered). The two LLM-fallback re-enablement attempts on Sep 17 reproduced this same failure shape live, but in both cases, disabling the LLM entirely and re-testing

proved the wrong answer was coming from the **deterministic compiler's own grounding gaps**, not the model. That's actually good news structurally: it means the fix is a general grounding/embedding-similarity signal, not "try a fourth model."

Why Phi-4-mini was not chosen as today's narration fix, despite finishing fastest

Phi-4-mini finished the benchmark narration request in 64s vs 72s for the Q4_K_M Qwen — marginally faster. But Phi-4-mini is a **different model family**, and the one time it was qualified for a production role (Sep 15, registered-operation selection) it failed outright and blew the latency budget 6x on the harder task. It has never been qualified for the *narration* role specifically — doing that safely means building a fresh grounding/citation-accuracy test corpus for it, which is not something to do for the first time the night before a demo. The Q4_K_M Qwen swap gets nearly the same speed win with zero new unknowns, because it's the same tokenizer, same prompt template, and same grounding behavior already validated across 711 tests.

Narration: what's fixed vs what's still open

Two real, structural bugs in the narration path were found and fixed (prior to today):

1. An empty JSON Schema enum: `[]` on templates with no citations was crashing llama.cpp's grammar sampler outright — every request of that shape got an HTTP 500.
2. Using full free-text sentences as JSON Schema enum values was catastrophically expensive for the grammar sampler — fixed by having the model choose short IDs (`L1` , `Q1` , ...) that get translated server-side, preserving "the model can only select, never invent."

Both fixes are unconditionally correct regardless of the latency question. But after both were fixed, the exact same production narration request still hit the same ~120-second ceiling, across four independent configuration changes (thread count, token budget, schema size) — the fact that the number barely moved was itself informative: it pointed at request-handling overhead, not the model. Today's live benchmark (raw model call: 72s finish time for Q4_K_M) versus the full pipeline result (119s, with a schema-validation fallback) confirms there is real overhead in the pipeline beyond raw model generation — narrowing that gap is the next concrete piece of work, not a mystery.

Dynamic runtime query engine: before / after

This is the direct answer to "queries not present in operations or predefined templates" — the exact gap named as the two-week blocker.

Before (early September): the ad-hoc/dynamic query path existed on paper (`forensics.source-native-plan/v1`) but was structurally incapable of answering correctly at real scale, for two compounding reasons:

1. `semanticFrameGoal` didn't recognize the word "many" as an aggregate signal, so "how many CDR records" was misclassified as a lookup and never even attempted a dynamic plan.
2. Once that was fixed, the dynamic path ran its entire computation **in memory over a capped 1,000-row sample** rather than as a real database aggregate — and hard-failed whenever the authorized scope exceeded that. The real demo case has 8,642 CDR rows, so this mechanism could not correctly answer a single real dynamic question at production scale.

After (shipped and live-verified today): a genuine SQL execution path (`source_native_sql_executor.go`) compiles the same validated plan contract into one parameterized SQL statement, executed by Postgres over the full authorized scope — every field name and literal passed as a bound parameter, never string-concatenated.

Concrete before/after, live-verified against the real demo case (`nexusai-forensic-demo` , 8,642 CDR rows):

Question	Before	After
"How many CDR records do we have in this case?"	Failed: <code>field_catalog_unavailable</code>	8,642 — independently checked against <code>SELECT count(*)</code> in Postgres, full citation lineage, under 300ms
"How many IPDR sessions..." (count)	HTTP 500 (field catalog sampled inconsistently between compile and execution)	2,500 — exact match, verified 3/3 repeated runs
"How many access log entries..."	Returned a 20-row raw listing instead of a count (intent-compatibility gap)	1,000 — exact DB match
"How many ANPR sightings..."	Same intent-gap symptom	750 — exact DB match

This was extended the same day across families: document search, image OCR, audio transcript search (including Roman-Urdu), and video observation timelines were all found to have a *separate*, deeper bug — natural-language search was returning irrelevant top-1

matches because the one relevance-filtering code branch that existed was unreachable (an unconditional `score=1` fallback made it dead code). Fixed and live-verified for OCR; spot-checked clean for the others.

What still doesn't work, stated plainly rather than hidden:

- Case-wide "who are the top talkers by volume" (rank without naming a specific target) has no registered or dynamic operation yet — the system now fails safely by asking for a target instead of guessing wrong, but a real "top talkers" operation needs to be designed and built separately.
- "How many records do we have for each record type" (a group-by variant) still routes through an older, separate mechanism and returns raw schema output instead of counts — flagged, not yet fixed.
- The keyword-routing fix addressed three known collision bugs structurally, but other collisions between a generic single word and a more specific phrase may still exist for pairs of operations not yet specifically tested.

Honest current status

Demo-ready today

- Dynamic runtime SQL queries across CDR, IPDR, ANPR, and access-log families — fast (under 300ms), exact, verified against the real database, not a template match.
- Document/OCR/audio-transcript/video search with working relevance filtering.
- Authentication is fail-closed and verified (401 without a token, 200 with a valid one).
- The full Go test suite: 711/711 specs passing as of the last fix cycle today.
- The real demo case data (`nexusai-forensic-demo` , tenant `default`) is present and intact: 8,642 CDR rows, matching every number verified throughout the log.

Not yet demo-ready — say this plainly if asked

- Open-ended narration ("explain what this suggests") can take 100+ seconds end-to-end and can still fall back to the deterministic answer under the new model, as it did in a live test today (fell back on a schema-validation failure, not a timeout this time — a different symptom, same underlying pipeline-overhead problem). **The safety mechanism worked correctly** — it never served an ungrounded answer — but the latency is not fixed, only measurably improved (raw model time dropped from "doesn't finish in 240s" to "72s").

- LLM-based query understanding (letting a model choose which operation answers a question) remains disabled in production. Every model tried for this role has failed; the deterministic compiler plus embedding-ranking is what actually resolves questions today.
- "Top talkers" case-wide ranking and the record-type group-by variant are known, named gaps — not fixed, not hidden.

Why this is a defensible two weeks of work, not a stall

The RAM-gated model evaluations (Sep 7–16) repeatedly hit real resource ceilings on this hardware before producing a quality verdict — that's evidence of a hardware/deployment constraint being taken seriously, not sloppy work being hidden. Two separate LLM-fallback attempts reproduced live wrong answers and were rolled back rather than shipped and hoped for — both times, the root cause was isolated and fixed at the deterministic layer instead of being left as a live bug. The dynamic-query engine — the actual headline ask — went from "structurally incapable of working at scale" to "exact, fast, and verified" in this final week, once the model evaluation work had exhausted the productive paths and pointed clearly at a deterministic-layer fix instead.

Demo script and novel test queries

Use collection `nexusai-forensic-demo` (tenant `default`) — confirmed live with 8,642 CDR rows plus IPDR/ANPR/access-log/document data. All queries below are phrased freshly for this brief and are not copied from any test fixture or predefined template list, so they genuinely exercise runtime understanding rather than a memorized match.

Tier 1 — lead with these (dynamic SQL engine, fast and exact, zero risk)

1. "How many CDR records do we have in this case?" → expect exactly **8,642**, under 300ms.
2. "How many IPDR sessions are in this case?" → expect exactly **2,500**.
3. "How many ANPR sightings do we have?" → expect exactly **750**.
4. "How many access log entries are there?" → expect exactly **1,000**.
5. "Show me the call type breakdown for 923461678183." → named target from the case notes; returns exact structured counts by call type.
6. "What's the earliest and latest call timestamp on record for this case?" — a genuinely novel aggregate not in the list above; good live test of the SQL executor's MIN/MAX path.

Tier 2 — safety-gate demonstrations (show the system refusing to guess)

7. "Who are the top callers in this case?" — no target named; correct behavior is a clarification request asking which number to analyze, not a guess. This demonstrates the intent-gating and grounding work directly — frame it as a feature, not a gap, when presenting.
8. "List the top 5 phone numbers by number of calls made." — this is the exact question that twice produced a live wrong-answer bug this session; it now correctly asks for a target. Good one to narrate the debugging story around if asked for a technical deep dive.

Tier 3 — cross-source / narrative (higher latency risk — test beforehand, have a fallback line ready)

9. "Show call type breakdown for 923461678183 and explain what it suggests." — open-ended narration; budget up to 120 seconds live, or pre-run it and show the captured result if presenting on a clock.
10. "Summarize the evidence for plate ABC-123 with source citations." — named in the case notes; exercises document + structured cross-referencing.
11. "Show data quality and duplicate-row counts for this case." — also named in the case notes; a good "we take data integrity seriously" closer.

Tier 4 — free-form, not in any corpus (use if asked to prove it isn't scripted)

12. "Which location appears most often across all CDR records?"
13. "Is there any call activity between midnight and 5am for 923461678183?"
14. "Find any document mentioning ABC-123."

Presentation tip: run Tier 1 and Tier 2 live — they are fast and 100% reliable. For Tier 3, either pre-run and show the transcript, or explicitly frame it as "this is the piece we're actively speeding up" before running it live, so a 60–120 second wait reads as an honest demo of in-progress work rather than a failure.

Anticipated questions and how to answer them

"Why did the LLM take two weeks and still isn't done?" Because three separate models were qualified against real, adversarial test corpora rather than accepted on a demo-friendly happy path, and two of those qualification cycles were blocked by real hardware RAM ceilings rather than model quality — that's a resource constraint on this deployment, not indecision. Every one of the last three model attempts (Qwen 4B full-plan, Qwen 4B registered-selection, Phi-4-mini) failed a live, scored qualification test rather than being

subjectively rejected. The two LLM-fallback re-enablement attempts this week each reproduced a real wrong answer live and were rolled back same-day rather than shipped — both root causes were found and fixed at the deterministic layer instead.

"So is the LLM working or not?" Narration works and is safe — it either produces a grounded, cited answer or it falls back to a verified deterministic one; it has never shipped an ungrounded or invented answer. It is not yet fast enough for a snappy live demo of open-ended questions (60–120s). Query understanding/planning by LLM is off in production; the deterministic compiler plus a small embedding-ranking assist is what actually resolves questions today, and it does so accurately.

"Why not just use a bigger/different model?" Tried: Qwen3-8B never got a fair evaluation because every attempt exhausted available RAM before completing enough cases — that's a hardware ceiling on the current host, not a decision to avoid it. Phi-4-mini was fully qualified and failed outright on the harder task (operation selection) with 6x the acceptable latency. The fix applied today (Q4_K_M quantization of the same model) is the correct low-risk lever precisely because the earlier benchmarking (thread count 8→16 made negligible difference) proved the bottleneck is memory bandwidth, not raw compute — a smaller model file, not a different model, is what that diagnosis calls for.

"What can you actually show me right now?" Exact, fast, verified answers to questions that touch no predefined template — counts, breakdowns, and targeted lookups across CDR/IPDR/ANPR/access-log data, computed via real parameterized SQL over the full dataset (8,642+ rows), not a template match and not a capped sample. That is the literal answer to "dynamic runtime queries not present in operations or predefined templates."

"Why should I trust these numbers weren't just made up for this meeting?" Every number in this brief was checked against the live system today: the model benchmark was run against the real production JSON-schema narration payload, the dynamic-query counts were cross-checked against `SELECT count(*)` in Postgres, and the auth fix was verified with both a fail-closed crash test and a real 401/200 request pair. Offer to re-run any Tier 1 or Tier 2 query live on request.

"What's the actual root cause of the narration slowness, if the model swap didn't fully fix it?" Two real bugs in the narration schema itself were found and fixed earlier (a grammar-sampler crash on empty enums, an expensive free-text-enum encoding) — those were unconditionally worth fixing regardless of speed. After both fixes, the same request still hit ~120s, and swapping to a smaller model quantization today cut the raw model-generation time from "doesn't finish in 240s" to 72s — but the full request pipeline still took 119s, meaning there is real overhead somewhere between the API receiving the request and the model call completing that hasn't been isolated yet. That is the next concrete, scoped piece of work — not an open-ended mystery.

Next steps

1. **Isolate the remaining narration pipeline overhead.** Add timestamp instrumentation around the HTTP client call inside `synthesizeFactPacketNarrativeWithTrace` to find where the gap between raw model time (72s) and full-request time (119s) is going — this was the specific next diagnostic step already identified in the engineering log and not yet executed.
2. **Replace curated-keyword grounding with a general embedding-similarity signal.** Two live LLM-fallback attempts each traced back to the same class of deterministic-compiler grounding gap; patching keywords one phrasing at a time will keep finding new gaps indefinitely. This was explicitly recommended in today's log as the productive next step over a third LLM re-attempt.
3. **Design and register a "top talkers" case-wide ranking operation.** Currently has no registered or dynamic path; the system safely asks for a target instead of guessing, but the capability itself doesn't exist yet.
4. **Route the record-type group-by question through the new SQL executor** instead of the older `canonical_group.go` mechanism it currently uses, which returns raw schema output instead of counts.
5. **Full keyword-routing audit** for other generic-word-vs-specific-phrase collisions beyond the three found and fixed this week.
6. **Re-attempt Qwen3-8B (or a comparable candidate) qualification with a hardened, RAM-aware runner** once host memory headroom allows a full run to complete — it has never actually received a fair quality evaluation, only resource-blocked attempts.

None of these require another two weeks — items 3–5 are scoped, bounded engineering tasks; item 1 is a single instrumented rebuild-and-test cycle; item 2 is the structural fix that the last two rollback cycles both pointed at independently.